SOFTWARE ENGINEERING · SPRINT 1

Escape Room Management System

Software Requirements Specification



[Team members] · [Date]

OVERVIEW

One system, two parts

Management system

Web dashboard and booking. Players reserve rooms; admins run the business and follow live games.

`Bun · Express · React · PostgreSQL`

Escape room client

The game itself: a 2D point-and-click room where the team solves puzzles against the clock.

`Unity · C# · REST API`

Both parts talk to the same backend, so bookings, games and scores live in one place.

FUNCTIONAL REQUIREMENTS

Two roles: Player and Admin

Player

- Registers, logs in and logs out
- Sees the available escape rooms
- Books a room and adds the team members
- Launches a game session straight away

Admin

- Dashboard with business data
- Past games and live games with progress and scores
- Room details: puzzles, steps and resources
- Manages, reorganizes and cancels bookings

FUNCTIONAL REQUIREMENTS

Inside the game

Puzzles

Clicking objects on screen shows the puzzle messages.

Hints

Players can ask for a hint or help during the session.

Leave early

An option to exit the room before the time runs out.

Final result. The score and completion time are calculated at the end and shown to the player in the game and to the admin on the dashboard.

NON-FUNCTIONAL REQUIREMENTS

Quality targets we can measure

CAPACITY

3

escape rooms tracked live at the same time

RESPONSE TIME

< 5 s

to load a game after the player clicks Start

AVAILABILITY

99.9%

uptime, so bookings are always open

CLIENT

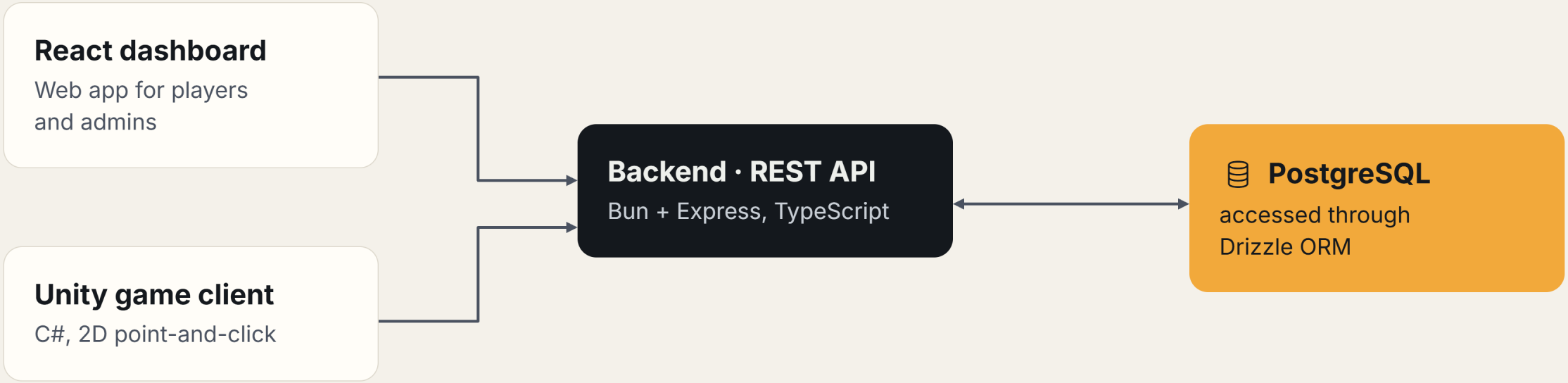
60

frames per second, at least, in the game

Vague goals such as “runs smoothly” become numbers that can be tested.

ARCHITECTURE

Architecture and tech stack



The clients send scores, hint requests and session checks to the backend over REST.

ESCAPE ROOM MANAGEMENT SYSTEM

Database model

PostgreSQL · Drizzle ORM · five tables



DATABASE MODEL

Five tables, one model

`user_table`
Players and admins

`map_table`
The escape rooms

`puzzles_table`
Four puzzles per room

`reservations`
Bookings for a room and date

`game_table`
Live and finished games

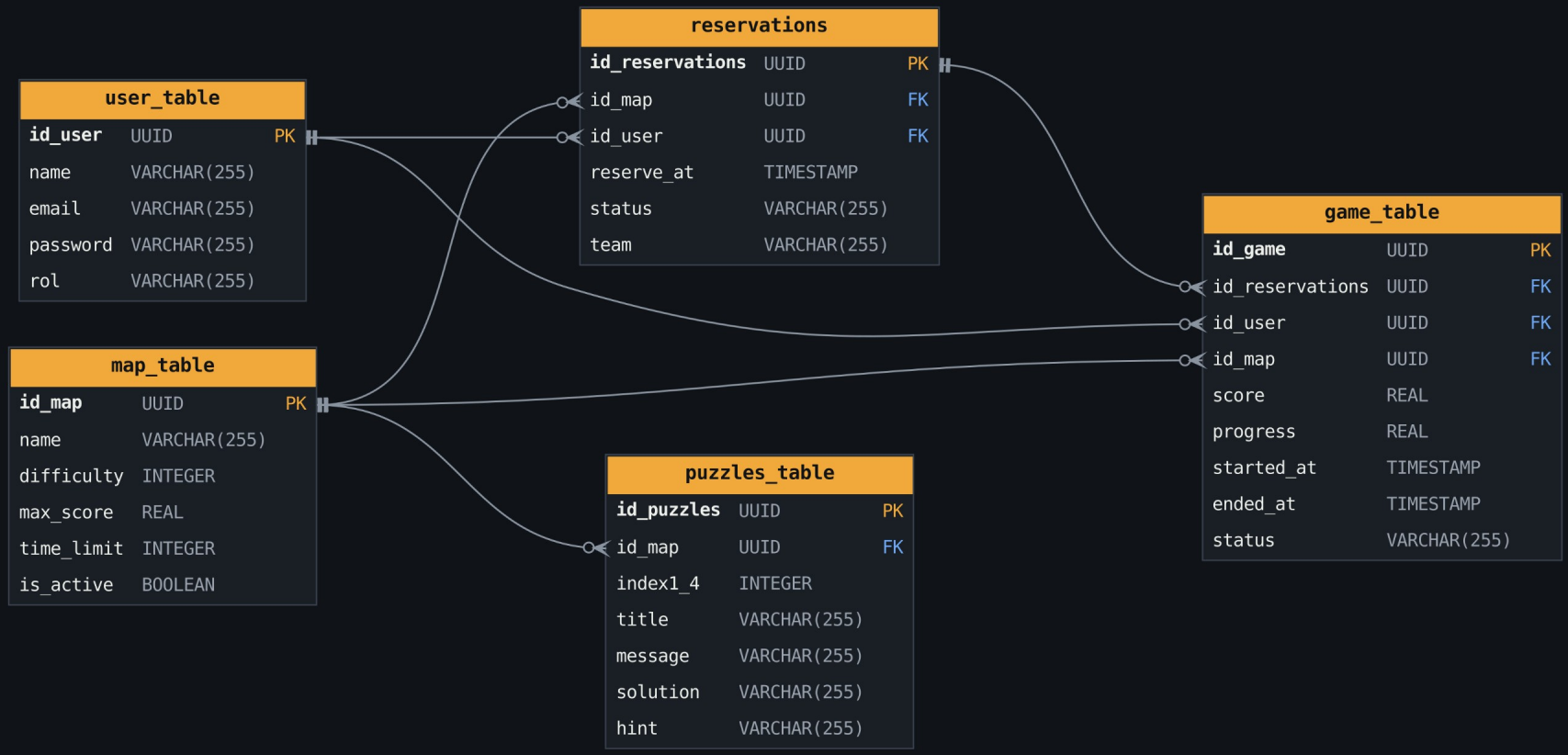
UUID primary key in every table

Stored in PostgreSQL

Defined in code with Drizzle ORM

DATABASE MODEL

Entity-relationship diagram



PK primary key · FK foreign key · bar = one · crow's foot = many

DATABASE MODEL · TABLE 1 OF 5

user_table

Everyone who uses the system. The role decides whether they see the player area or the admin dashboard.

RELATIONSHIPS

1 : N **reservations**
A user makes many bookings

1 : N **game_table**
A user plays many games

Field	Type	Description
id_user	UUID · PK	Unique user ID
name	VARCHAR(255)	Full name
email	VARCHAR(255)	Login email, must be unique
password	VARCHAR(255)	Stored as a hash, never in plain text
rol	VARCHAR(255)	admin or player

DATABASE MODEL · TABLE 2 OF 5

map_table

Each escape room (map) the business offers, with its difficulty and time limit.

RELATIONSHIPS

1 : N **puzzles_table**
A room is made of four puzzles

1 : N **reservations**
A room is booked many times

1 : N **game_table**
A room hosts many games

Field	Type	Description
id_map	UUID · PK	Unique room ID
name	VARCHAR(255)	Room name
difficulty	INTEGER	Difficulty level
max_score	REAL	Highest score a team can get
time_limit	INTEGER	Time limit, in minutes
is_active	BOOLEAN	Whether it can be booked now

DATABASE MODEL · TABLE 3 OF 5

puzzles_table

The puzzles inside each room. Every room has four, solved in order from 1 to 4.

RELATIONSHIPS

N : 1 **map_table**
Each puzzle belongs to one room

Field	Type	Description
id_puzzles	UUID · PK	Unique puzzle ID
id_map	UUID · FK	Room it belongs to
index1_4	INTEGER	Position in the room, 1 to 4
title	VARCHAR(255)	Puzzle name
message	VARCHAR(255)	Text shown when the object is clicked
solution	VARCHAR(255)	Correct answer
hint	VARCHAR(255)	Optional clue for the player

DATABASE MODEL · TABLE 4 OF 5

reservations

The bookings players make for a room on a given date and time.

RELATIONSHIPS

N : 1 **user_table**
Made by one player

N : 1 **map_table**
For one room

1 : N **game_table**
Games are created from it

Field	Type	Description
id_reservations	UUID · PK	Unique booking ID
id_map	UUID · FK	Booked room
id_user	UUID · FK	Player who booked
reserve_at	TIMESTAMP	Date and time of the session
status	VARCHAR(255)	pending, confirmed, cancelled or completed
team	VARCHAR(255)	Team members (optional)

DATABASE MODEL · TABLE 5 OF 5

game_table

A game played from a booking. It feeds the live dashboard and keeps the final result.

RELATIONSHIPS

N : 1 **reservations**
Starts from one booking

N : 1 **user_table**
Started by one player

N : 1 **map_table**
Played in one room

Field	Type	Description
id_game	UUID · PK	Unique game ID
id_reservations	UUID · FK	Booking it comes from
id_user	UUID · FK	Player who started it
id_map	UUID · FK	Room being played
score	REAL	Live score, then final score
progress	REAL	% of the room completed
started_at	TIMESTAMP	Start time
ended_at	TIMESTAMP	End time, empty while playing
status	VARCHAR(255)	in progress, completed or abandoned

DATABASE MODEL

Six one-to-many relationships

One	Many	Type	Meaning
user_table	reservations	1 : N	A user makes many bookings
map_table	reservations	1 : N	A room is booked many times
map_table	puzzles_table	1 : N	A room holds its four puzzles
reservations	game_table	1 : N	Every game starts from a booking
user_table	game_table	1 : N	A user plays many games
map_table	game_table	1 : N	A room hosts many games

Each link is a foreign key (FK) on the “many” side that points to the primary key (PK) of the “one” side.

DATABASE MODEL

How the data flows

01

Sign up

`user_table`

A player creates an account and logs in.

02

Book

`reservations`

They pick a room and a date and add their team.

03

Play

`game_table`

Starting the session creates a game that tracks progress and score.

04

Result

`game_table`

Score and time are saved and shown to the player and the admin.

Behind every game, the room (**map_table**) supplies its four puzzles (**puzzles_table**).

Questions?

